Week 12 Asynchronous Materials Guide - Functions

This Week's Focus

Building on lecture: The lecture introduced function basics. This guide expands into variable scope, default parameters, and common patterns.

- Review: Creating functions, parameters, return values
- New: Variable scope (local vs global)
- New: Default parameter values
- New: Common function patterns

Start Here - 20 Minutes

Video: Python Functions

Corey Schafer - Python Functions

- Covers def, parameters, return values
- Clear explanations with examples

Core References

W3Schools - Python Functions

https://www.w3schools.com/python/python_functions.asp

- Quick reference for syntax
- Interactive examples

Programiz - Python Functions

https://www.programiz.com/python-programming/function

- Clear explanations with diagrams
- Practice examples included

Review: Function Basics

Covered in lecture. Quick review before new material.

Creating and Calling Functions

```
# Define a function
def greet(name):
    print(f"Hello, {name}")

greet("Alice")

# Function with return
def add(a, b):
    return a + b

result = add(5, 3)
print(result)  # 8
```

Variable Scope

Variables created inside a function only exist inside that function.

Local vs Global Variables

```
# Local variables
def calculate_total():
    price = 10  # Only exists inside function
    quantity = 5
    total = price * quantity
    print(f"Total: {total}")

calculate_total()
```

```
# print(price)  # Error - doesn't exist outside

# Global variables
tax_rate = 0.08

def calculate_with_tax(price):
    tax = price * tax_rate  # Can read global
    return price + tax

print(calculate_with_tax(100))
```

Best Practice: Pass Data as Parameters

```
# Avoid relying on globals
tax_rate = 0.08
def calculate_total(price):
    return price * (1 + tax_rate)

# Better - explicit parameters
def calculate_total(price, tax_rate):
    return price * (1 + tax_rate)

total = calculate_total(100, 0.08)
```

Default Parameter Values

Using Defaults

```
# Default parameter
def greet(name, greeting="Hello"):
    print(f"{greeting}, {name}")

greet("Alice")  # Uses default
greet("Bob", "Hi")  # Custom greeting

# Multiple defaults
def calculate_price(base, tax_rate=0.08, discount=0):
    subtotal = base * (1 + tax_rate)
    return subtotal * (1 - discount)
```

```
print(calculate_price(100))
print(calculate_price(100, 0.10))
print(calculate_price(100, 0.08, 0.15))
```

Common Function Patterns

Pattern 1: Validation

```
# Return True/False
def is_valid_email(email):
    return "@" in email and "." in email

def is_in_range(value, min_val, max_val):
    return min_val <= value <= max_val

# Use in conditionals
if is_valid_email("user@example.com"):
    print("Valid email")

if is_in_range(85, 0, 100):
    print("Valid score")
```

Pattern 2: Transform

```
# Take input, return transformed output
def format_price(price):
    return f"${price:.2f}"

def celsius_to_fahrenheit(celsius):
    return (celsius * 9/5) + 32

print(format_price(19.99))
print(celsius_to_fahrenheit(25))
```

Pattern 3: Aggregation

```
# Process collection, return summary
def count_passing(grades):
    count = 0
    for grade in grades:
        if grade >= 60:
```

```
            count += 1
    return count

def find_highest(numbers):
    highest = numbers[0]
    for num in numbers:
        if num > highest:
            highest = num
    return highest

scores = [85, 55, 92, 78, 45]
print(f"Passing: {count_passing(scores)}")
```

Pattern 4: Filter

```
# Return subset based on criteria
def get_passing_scores(scores):
    passing = []
    for score in scores:
        if score >= 60:
            passing.append(score)
    return passing

all_scores = [85, 55, 92, 78, 45, 88]
passing = get_passing_scores(all_scores)
print(f"Passing: {passing}")
```

Common Mistakes

Mistake 1: Forgetting to Return

```
# WRONG - prints but doesn't return
def add(a, b):
    result = a + b
    print(result)

total = add(5, 3)  # total is None

# CORRECT
def add(a, b):
    return a + b
```

```
total = add(5, 3)  # total is 8
```

Mistake 2: Return Exits Function

```
# WRONG - stops after first match
def process_scores(scores):
    for score in scores:
        if score >= 90:
            return "Found an A"
    return "No A's"

# CORRECT - collect all results
def process_scores(scores):
    a_grades = []
    for score in scores:
        if score >= 90:
            a_grades.append(score)
    return a_grades
```

Quick Reference

Function Basics

```
# Define
def function_name(parameter1, parameter2):
    return value

# Call
result = function_name(arg1, arg2)
```

Key Concepts

```python
# Local scope
def example():
    local_var = 10  # Only inside function

# Default parameters
def greet(name, greeting="Hello"):
    print(f"{greeting}, {name}")

# Common patterns
def validate(value):
    return condition

def transform(input):
    return modified

def aggregate(collection):
    # process
    return summary

def filter_items(collection):
    filtered = []
    for item in collection:
        if condition:
            filtered.append(item)
    return filtered
```